RoadPulse — Product Requirements Document

Product: RoadPulse
Version: 1.0 — Hackathon MVP
Category: Road safety / Civic technology
Platform: Responsive web application
Primary users: Citizens, safety reviewers, road safety administrators

1. Product vision

RoadPulse is a near-miss intelligence and road-safety prioritization platform. It collects structured observations of dangerous road situations, combines nearby observations into location-level risk signals, and helps reviewers prioritize field audits and track safety interventions.

The product is an early-warning and decision-support tool, not a guaranteed accident predictor or official blackspot identification system.

2. Problem statement

Road users experience dangerous situations that do not necessarily result in recorded accidents. Examples include sudden braking at blind turns, pedestrian near-collisions, poor visibility, unsafe crossings and recurring vehicle-pedestrian conflicts.

RoadPulse aims to make these observations structured, geographically discoverable and actionable.

3. Goals

- Collect structured near-miss reports with location, time and hazard type.
- Display observations on an interactive map.
- Group nearby reports into location-level clusters.
- Prioritize locations using explainable, configurable rules.
- Enable report verification and safety-audit assignment.
- Track interventions and subsequent observations.

Non-goals

- Exact accident prediction.
- Automatic official blackspot designation.
- Unauthorised integration with government systems.
- Real-time vehicle tracking.
- Automatic emergency dispatch.

4. User roles

Citizen: Submit near-miss observations, view public hazards and corroborate existing reports.

Safety reviewer: Review evidence, mark duplicates, verify observations and update review status.

Administrator: Manage review workflows, assign audits, configure scoring rules and monitor outcomes.

5. Functional requirements

FR-01: Near-miss reporting

Users submit hazard category, location, incident date/time, road-user type and description. Photos are optional.
Acceptance criteria:

- Location can be selected on a map or through user-authorized device location.
- Required fields are validated.
- Each report has a unique ID.
- Incident time and submission time are stored separately.

FR-02: Interactive map

Show report markers with category, date range, verification and priority filters.
Acceptance criteria:

- Selecting a marker displays a report summary.
- Nearby reports can be explored together.
- Unverified reports are visibly distinguished.
- Personal information is not exposed on public maps.

FR-03: Risk prioritization

Calculate a configurable location-level screening score using repetition, potential severity, recency, evidence quality and independent corroboration.
Acceptance criteria:

- Each score has a human-readable explanation.
- Reviewers can inspect contributing reports.
- Missing data is handled explicitly.
- Scores are labelled as screening indicators, not accident probabilities.

FR-04: Verification workflow

Authorized reviewers can mark reports as under review, verified, rejected or duplicate.
Acceptance criteria:

- Only authorized roles can change verification status.
- Decisions and timestamps are recorded.
- Audit tasks have an owner, due date and status.

FR-05: Safety-audit tracking

Create an audit task for a selected location and record findings, proposed interventions, completion dates and follow-up observations.

FR-06: Outcome monitoring

Compare observations before and after an intervention using clearly defined time windows. Do not infer causation from report-count changes alone.

6. Risk scoring model

Proposed initial weights, to be calibrated during testing:

- Repeated observations: 30%
- Potential severity: 25%
- Recency: 20%
- Evidence quality: 15%
- Independent corroboration: 10%

Each factor is normalized from 0 to 100.
Score = 0.30R + 0.25V + 0.20T + 0.15E + 0.10C

Suggested display bands:

- 0–39: Routine review
- 40–69: Elevated review priority
- 70–100: High review priority

These are proposed product thresholds, not validated safety standards. Scores must be explainable, configurable and subject to human review. Duplicate or coordinated submissions must not automatically increase priority.

7. User workflow

1. Citizen submits a near-miss report.
2. Backend validates required fields and flags potential duplicates.
3. Reports are grouped by geographic proximity.
4. The scoring engine calculates a transparent screening score.
5. A reviewer verifies evidence and prioritizes a field audit.
6. An administrator records a proposed or completed intervention.
7. Follow-up observations are compared with the baseline period.

8. Technology stack

- Frontend: React and Tailwind CSS.
- Backend: Node.js and Express.
- Database: PostgreSQL with PostGIS.
- Mapping: Leaflet and OpenStreetMap.
- Charts: Chart.js.
- Evidence storage: Supabase Storage or equivalent.
- Authentication: Supabase Auth or secure backend-managed authentication.

No custom AI/ML model is required.

9. Core data entities

Users: id, role, created_at.
Reports: id, category, description, latitude, longitude, incident_at, submitted_at, status.
Evidence: id, report_id, file_path, uploaded_at.
Locations: id, geometry, risk_score, score_updated_at.
Reviews: id, report_id, reviewer_id, decision, notes.
Audits: id, location_id, assigned_to, due_date, status.
Interventions: id, location_id, action, completed_at.
Observations: id, location_id, observed_at, observation_type.

10. Non-functional requirements

- Role-based access control and protected uploads.
- Rate limiting and input validation.
- Data minimization and privacy-conscious map displays.
- Responsive, accessible user interface.
- Paginated report lists and viewport-based map loading.
- Audit logs for review decisions and score updates.
- Clear distinction between reported, verified and resolved information.

11. Success metrics

- Percentage of reports with complete required fields.

- Percentage of reports reviewed.
- Median submission-to-review time.
- Number of locations with independent corroboration.
- Number of audit tasks completed.
- Change in near-miss reports during defined pre- and post-intervention periods.

Report counts are affected by participation and reporting coverage. They must not be presented as direct proof of accident reduction.

12. Ten-day implementation plan

Days 1–2: Project setup, authentication and database schema.
Days 3–4: Report form, location selection, evidence upload and report list.
Days 5–6: Geographic grouping, scoring rules and map filters.
Days 7–8: Reviewer dashboard, audit tasks and activity logs.
Days 9–10: Synthetic demo data, testing, UI refinement and presentation.

13. Risks and mitigations

- False reports: rate limiting, duplicate flags and human review.
- Limited real-world data: use clearly labelled synthetic data for the demo.
- Lack of official integration: demonstrate internal audit workflows.
- Misleading scores: expose scoring factors and require field verification.
- Privacy concerns: minimize personal data and restrict evidence access.
- Unproven impact: track interventions without claiming causation.

14. Demo scenario

A citizen reports a near-collision at an intersection. Two independent reports appear nearby. RoadPulse groups the observations, explains the elevated priority score, and enables a reviewer to create a safety-audit task. An administrator records a proposed intervention, and follow-up observations are used to assess whether the reported hazard persists.

Use synthetic data for the hackathon demo and label it accordingly. A real pilot should use consented, manually verified observations and appropriate safety review.

15. Product boundary

RoadPulse complements existing road-safety reporting and accident-data systems. It does not replace official blackspot criteria, professional road-safety audits, emergency services or government authority decisions.